

NEHA SHARMA U24AI050 LAB 10

30 March 2026 21:44

1. To Write a C program to simulate producer-consumer problem using semaphores.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int item = 1;

    while (1) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d at %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        item++;
        sleep(1);
    }
}

void* consumer(void* arg) {
    while (1) {
        sem_wait(&full);
```

```

    pthread_mutex_lock(&mutex);

    int item = buffer[out];
    printf("Consumed: %d from %d\n", item, out);
    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);
    sem_post(&empty);

    sleep(2);
}
}

int main() {
    pthread_t p, c;

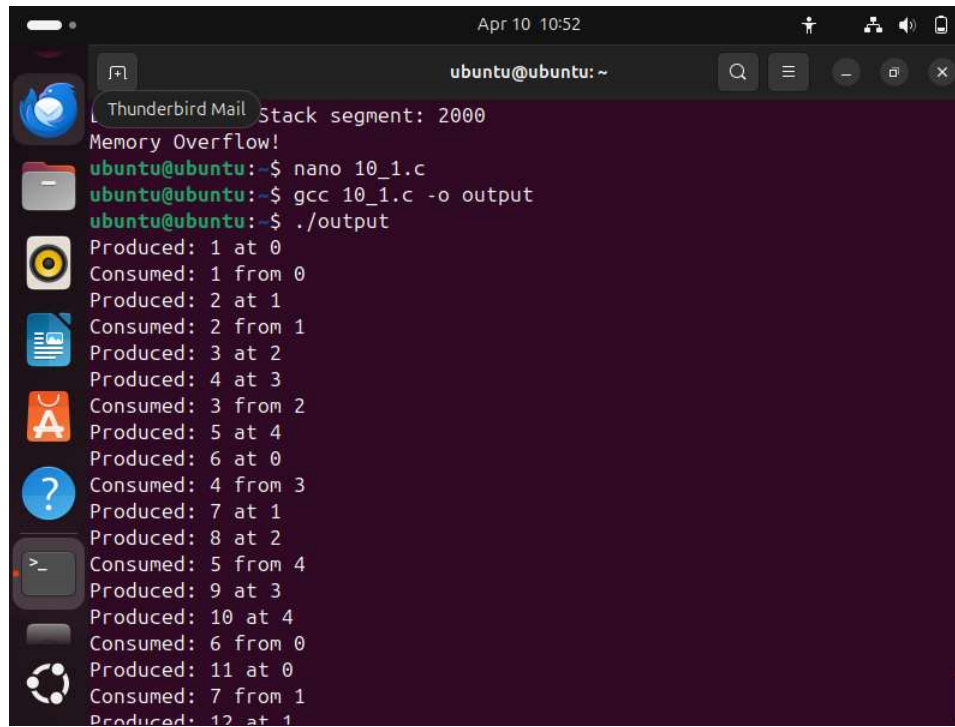
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    return 0;
}

```



```
ubuntu@ubuntu: ~  
Stack segment: 2000  
Memory Overflow!  
ubuntu@ubuntu:~$ nano 10_1.c  
ubuntu@ubuntu:~$ gcc 10_1.c -o output  
ubuntu@ubuntu:~$ ./output  
Produced: 1 at 0  
Consumed: 1 from 0  
Produced: 2 at 1  
Consumed: 2 from 1  
Produced: 3 at 2  
Produced: 4 at 3  
Consumed: 3 from 2  
Produced: 5 at 4  
Produced: 6 at 0  
Consumed: 4 from 3  
Produced: 7 at 1  
Produced: 8 at 2  
Consumed: 5 from 4  
Produced: 9 at 3  
Produced: 10 at 4  
Consumed: 6 from 0  
Produced: 11 at 0  
Consumed: 7 from 1  
Produced: 12 at 1
```

2. Write a C program to simulate the concept of Dining-Philosophers problem.

```
#include <stdio.h>  
#include <pthread.h>  
#include <semaphore.h>  
#include <unistd.h>  
  
#define N 5  
  
sem_t chopstick[N];  
  
void* philosopher(void* num) {  
    int id = *(int*)num;  
  
    while (1) {  
        printf("Philosopher %d is thinking\n", id);  
        sleep(1);
```

```

    sem_wait(&chopstick[id]);
    sem_wait(&chopstick[(id + 1) % N]);

    printf("Philosopher %d is eating\n", id);
    sleep(2);

    sem_post(&chopstick[id]);
    sem_post(&chopstick[(id + 1) % N]);
}
}

int main() {
    pthread_t ph[N];
    int id[N];

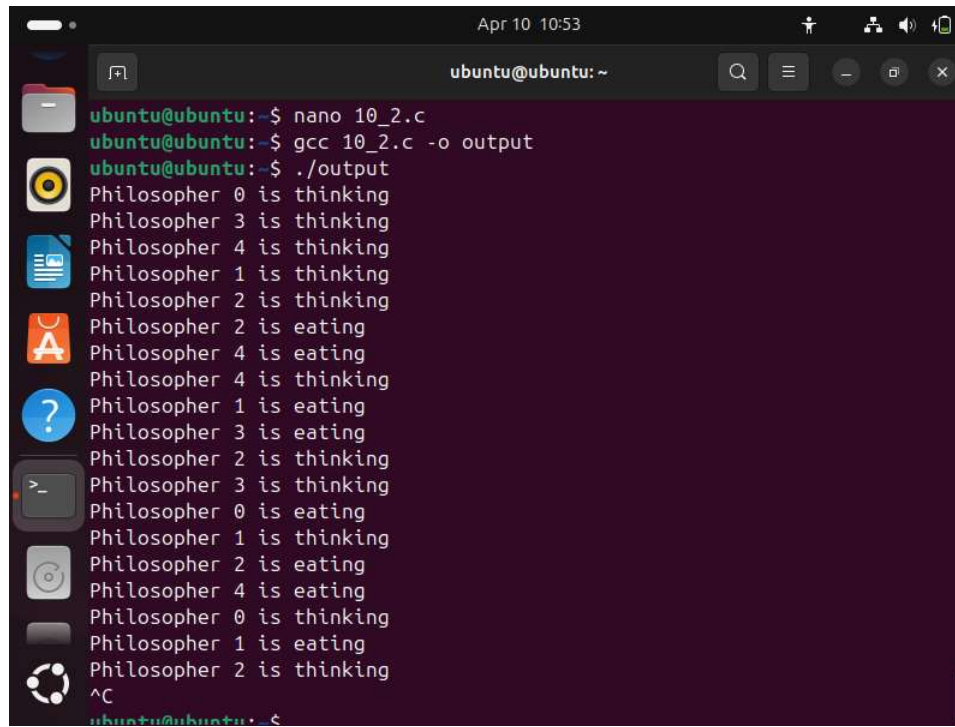
    for (int i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);

    for (int i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&ph[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(ph[i], NULL);

    return 0;
}

```



A terminal window on Ubuntu showing the execution of a C program. The user runs `nano 10_2.c`, `gcc 10_2.c -o output`, and `./output`. The program output shows five philosophers (0-4) alternating between thinking and eating. The sequence of actions is: Philosopher 0 thinks, Philosopher 3 thinks, Philosopher 4 thinks, Philosopher 1 thinks, Philosopher 2 thinks, Philosopher 2 eats, Philosopher 4 eats, Philosopher 4 thinks, Philosopher 1 eats, Philosopher 3 eats, Philosopher 2 thinks, Philosopher 3 thinks, Philosopher 0 eats, Philosopher 1 thinks, Philosopher 2 eats, Philosopher 4 eats, Philosopher 0 thinks, Philosopher 1 eats, and Philosopher 2 thinks. The program is interrupted with `^C`.

```
ubuntu@ubuntu: ~  
ubuntu@ubuntu:~$ nano 10_2.c  
ubuntu@ubuntu:~$ gcc 10_2.c -o output  
ubuntu@ubuntu:~$ ./output  
Philosopher 0 is thinking  
Philosopher 3 is thinking  
Philosopher 4 is thinking  
Philosopher 1 is thinking  
Philosopher 2 is thinking  
Philosopher 2 is eating  
Philosopher 4 is eating  
Philosopher 4 is thinking  
Philosopher 1 is eating  
Philosopher 3 is eating  
Philosopher 2 is thinking  
Philosopher 3 is thinking  
Philosopher 0 is eating  
Philosopher 1 is thinking  
Philosopher 2 is eating  
Philosopher 4 is eating  
Philosopher 0 is thinking  
Philosopher 1 is eating  
Philosopher 2 is thinking  
^C  
ubuntu@ubuntu:~$
```

3. Write a C program to simulate the concept of Readers-Writers problem.

```
#include <stdio.h>  
#include <pthread.h>  
#include <semaphore.h>  
#include <unistd.h>  
  
sem_t wrt;  
pthread_mutex_t mutex;  
  
int readcount = 0;  
  
void* reader(void* arg) {  
    int id = *(int*)arg;  
  
    while (1) {  
        pthread_mutex_lock(&mutex);  
        readcount++;  
        if (readcount == 1) {  
            sem_wait(&wrt);  
        }  
        printf("Reader %d is reading\n", id);  
        sleep(1);  
        pthread_mutex_unlock(&mutex);  
    }  
}
```

```

    if (readcount == 1)
        sem_wait(&wrt);
    pthread_mutex_unlock(&mutex);

    printf("Reader %d is reading\n", id);
    sleep(1);

    pthread_mutex_lock(&mutex);
    readcount--;
    if (readcount == 0)
        sem_post(&wrt);
    pthread_mutex_unlock(&mutex);

    sleep(1);
}
}

void* writer(void* arg) {
    int id = *(int*)arg;

    while (1) {
        sem_wait(&wrt);

        printf("Writer %d is writing\n", id);
        sleep(2);

        sem_post(&wrt);
        sleep(1);
    }
}

int main() {
    pthread_t r[3], w[2];
    int id[3] = {1,2,3};

    sem_init(&wrt, 0, 1);
    pthread_mutex_init(&mutex, NULL);

    for (int i = 0; i < 3; i++)
        pthread_create(&r[i], NULL, reader, &id[i]);

```

```

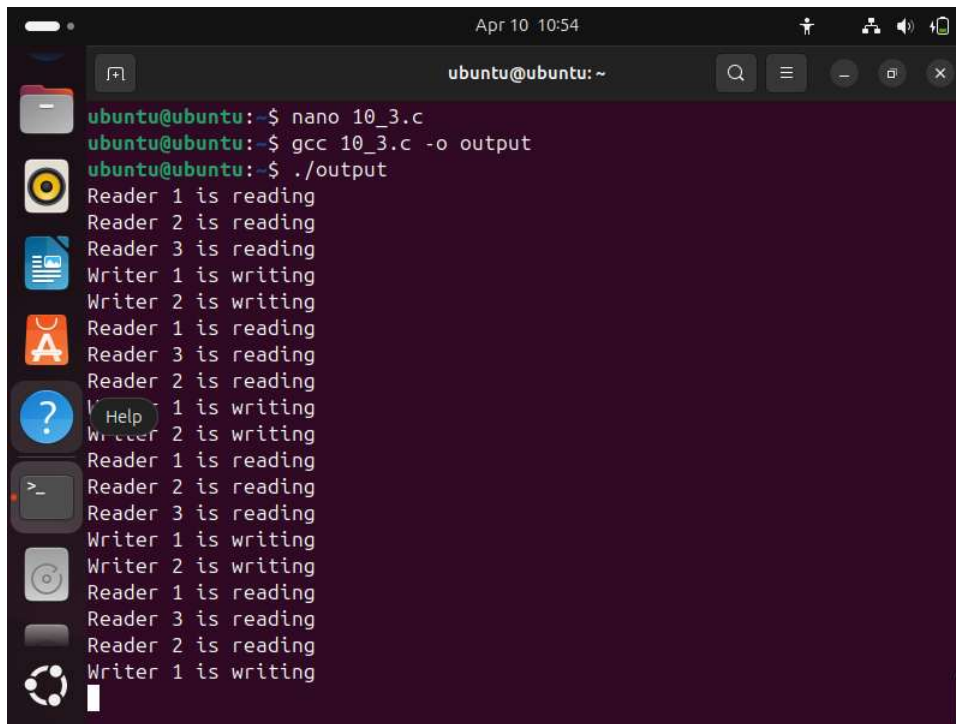
for (int i = 0; i < 2; i++)
    pthread_create(&w[i], NULL, writer, &id[i]);

for (int i = 0; i < 3; i++)
    pthread_join(r[i], NULL);

for (int i = 0; i < 2; i++)
    pthread_join(w[i], NULL);

return 0;
}

```



```

ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ nano 10_3.c
ubuntu@ubuntu:~$ gcc 10_3.c -o output
ubuntu@ubuntu:~$ ./output
Reader 1 is reading
Reader 2 is reading
Reader 3 is reading
Writer 1 is writing
Writer 2 is writing
Reader 1 is reading
Reader 3 is reading
Reader 2 is reading
Writer 1 is writing
Writer 2 is writing
Reader 1 is reading
Reader 2 is reading
Reader 3 is reading
Writer 1 is writing
Writer 2 is writing
Reader 1 is reading
Reader 3 is reading
Reader 2 is reading
Writer 1 is writing

```